

PROJECT GYM

Desenvolvimento Web II

Grupo: **inf25dw2g10**

Felipe Castilho	-	a047152
Juliana Moreira	-	a047188
Marta Vieira	-	a046756



Introdução

Este trabalho tem como objetivo desenvolver uma camada de **serviços REST**, protegida por uma camada de **Autenticação e Autorização** para disponibilizar os recursos para uma aplicação Web.

A plataforma desenvolvida permite que **admins**, **treinadores** e **clientes** possam criar e gerir planos de treino, definir os exercícios incluídos em cada plano e registar as sessões de treino realizadas, registar avaliações e metas, entre outros.



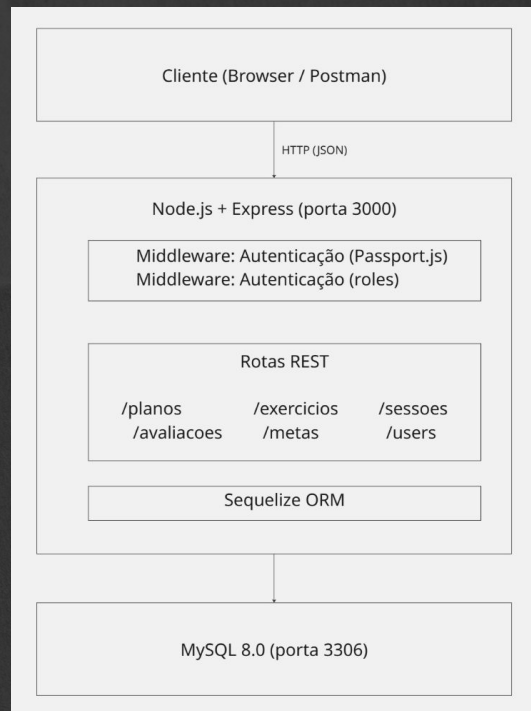
Arquitetura

API segue uma arquitetura em camadas com separação clara de responsabilidades.

As *Routes* definem os endpoints e encaminham os pedidos para os *Controllers*, que concentram a lógica de cada recurso.

O *Middleware* trata das regras de segurança, garantindo a autenticação (OAuth2, API Key ou Basic Auth) e autorização por perfil com uma função *ensureRole* (admin, treinador, cliente).

A camada de *Models* representa as entidades e relações no *Sequelize*, enquanto a *Config* faz a ligação entre parâmetros da base de dados *MySQL*.



Tabelas principais

As tabelas principais do sistema são: *Utilizadores*, *PlanosTreino*, *Exercicios*, *Sessões*, *Avaliacoes Fisicas* e *Metas*.

Estas foram modeladas com Sequelize para manter a estrutura de dados organizada e relacional.

Utilizadores é a entidade central (admin, treinador e cliente) e liga-se às restantes tabelas conforme o papel no domínio.

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Utilizador = sequelize.define('Utilizador', {
  id: {
    type: DataTypes.INTEGER,
    autoIncrement: true,
    primaryKey: true
  },
  providerId: {
    type: DataTypes.STRING(100),
    allowNull: false,
    unique: true,
    field: 'provider_id'
  },
  provider: {
    type: DataTypes.ENUM('github', 'google', 'basic'),
    allowNull: false,
    defaultValue: 'github'
  },
  username: {
    type: DataTypes.STRING(100),
    allowNull: false
  },
  displayName: {
    type: DataTypes.STRING(150),
    allowNull: true,
    field: 'display_name'
  },
  email: {
    type: DataTypes.STRING(200),
    allowNull: true
  },
  passwordHash: {
    type: DataTypes.STRING(255),
    allowNull: true,
  }
});
```

Tabelas principais

PlanosTreino organiza os planos de cada cliente/treinador e serve de base para os *Exercicios*, *Sessões* e *Metas*.

AvaliaçõesFísicas guarda registos de evolução física do cliente.

Com o Sequelize, além de mapear as tabelas, foram definidas associações (*hasMany/belongsTo*) de 1 para muitos.

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Utilizador = sequelize.define('Utilizador', {
  id: {
    type: DataTypes.INTEGER,
    autoIncrement: true,
    primaryKey: true
  },
  providerId: {
    type: DataTypes.STRING(100),
    allowNull: false,
    unique: true,
    field: 'provider_id'
  },
  provider: {
    type: DataTypes.ENUM('github', 'google', 'basic'),
    allowNull: false,
    defaultValue: 'github'
  },
  username: {
    type: DataTypes.STRING(100),
    allowNull: false
  },
  displayName: {
    type: DataTypes.STRING(150),
    allowNull: true,
    field: 'display_name'
  },
  email: {
    type: DataTypes.STRING(200),
    allowNull: true
  },
  passwordHash: {
    type: DataTypes.STRING(255),
    allowNull: true,
  }
});
```

Roles e Permissões

Planos de treino

Existem dois tipos: pessoal (cliente) e profissional (treinador).

Cliente:

- Pode criar/editar/apagar planos pessoais próprios;
- Não pode associar treinador ao plano pessoal;
- Não pode criar/editar/apagar planos de outros clientes.

Treinador:

- Pode criar/editar/apagar apenas planos profissionais em que é o *treinadorId*;
- Não pode alterar planos pessoais de clientes;
- Não pode operar sobre planos de outro treinador.



Roles e Permissões

Exercícios

Todos os exercícios pertencem a um plano.

Cliente:

Em plano pessoal: pode criar/editar/apagar;

Em plano profissional: pode atualizar *series*, *reps*, *pesoKg*, *notas*;

Não pode operar exercícios de outros clientes.

Treinador:

Só opera exercícios em planos profissionais dos quais é o treinador responsável;

Não opera exercícios de planos de outros treinadores/clientes.



Roles e Permissões

Sessões

Sessões pertencem a plano e representam aulas com treinador.
Criação: apenas admin e treinador.

Cliente:

Não pode criar nem apagar sessões;
Só pode editar notas em sessão própria.

Treinador:

Cria/edita/apaga sessões em que é o treinador responsável.



Roles e Permissões

Avaliações físicas

2 tipos de avaliações: pessoal (cliente) e profissional (treinador/admin).

Cliente:

- Pode consultar avaliações próprias;
- Pode criar/editar/apagar apenas avaliações pessoais próprias;
- Não pode editar/apagar avaliações profissionais.

Treinador:

- Cria avaliações profissionais para clientes permitidos
- Edita/apaga apenas avaliações profissionais por si criadas



Roles e Permissões

Metas

Metas estão sempre ligadas a um plano.

Cliente:

Cria metas apenas com *planold* de plano pessoal próprio;
Pode editar/apagar metas apenas de plano pessoal próprio.

Treinador:

Cria/edita/apaga metas apenas em planos profissionais do seu âmbito.



Roles e Permissões

Utilizadores

Cliente:

Sem acesso a *GET /users* e *PUT /users/{id}/role*;
Acesso a *GET /users/me* e *POST /users/me/api-key*.

Treinador:

GET /users devolve clientes do seu âmbito e também clientes sem plano profissional associado a treinador;
Sem acesso a *PUT /users/{id}/role*.

Admin:

Acesso completo aos utilizadores.



Relações entre Recursos

Utilizadores → Planos de Treino

Um treinador cria vários planos; um cliente tem vários planos atribuídos

Planos de Treino → Exercícios

Um plano tem múltiplos exercícios; um exercício pertence a um único plano

Planos de Treino → Sessões

Um plano tem múltiplas sessões registadas

Utilizadores → Sessões

Um cliente realiza várias sessões; um treinador supervisiona várias sessões

```
Utilizador.hasMany(PlanoTreino, { foreignKey: 'clienteId', as: 'planosComoCliente' });
PlanoTreino.belongsTo(Utilizador, { foreignKey: 'clienteId', as: 'cliente' });

PlanoTreino.hasMany(Exercicio, { foreignKey: 'planoId', as: 'exercicios', onDelete: 'CASCADE' });
Exercicio.belongsTo(PlanoTreino, { foreignKey: 'planoId', as: 'plano' });
```

Relações entre Recursos

Utilizadores → Avaliações Físicas

Um cliente tem várias avaliações físicas

Utilizadores → Metas

Um cliente tem muitas metas definidas

Planos de Treino → Metas

Um plano pode ter várias metas associadas

```
Utilizador.hasMany(AvaliacaoFisica, {  
  AvaliacaoFisica.belongsTo(Utilizador,
```

```
Utilizador.hasMany(Meta, { foreignKey  
Meta.belongsTo(Utilizador, { foreignKey
```

```
PlanoTreino.hasMany(Meta, { foreignKey  
Meta.belongsTo(PlanoTreino, { foreignKey
```

Endpoints Disponíveis

Autenticação

POST /auth/register Registrar utilizador com email e password

POST /auth/login Login com email/username e password

GET /auth/logout Terminar sessão

Planos de Treino

GET /planos Listar planos de treino

POST /planos Criar plano de treino

GET /planos/{id} Obter plano por ID

PUT /planos/{id} Atualizar plano

DELETE /planos/{id} Apagar plano

Exercícios

GET /exercicios Listar exercicios

POST /exercicios Criar exercicio

GET /exercicios/{id} Obter exercicio por ID

PUT /exercicios/{id} Atualizar exercicio

DELETE /exercicios/{id} Apagar exercicio

Sessões

GET /sessoes Listar sessões

POST /sessoes Criar sessão

GET /sessoes/{id} Obter sessão por ID

PUT /sessoes/{id} Atualizar sessão

DELETE /sessoes/{id} Apagar sessão

Avaliações Físicas

GET /avaliacoes Listar avaliações físicas

POST /avaliacoes Criar avaliação física

GET /avaliacoes/{id} Obter avaliação por ID

PUT /avaliacoes/{id} Atualizar avaliação

DELETE /avaliacoes/{id} Apagar avaliação

Metas

GET /metas Listar metas

POST /metas Criar meta

GET /metas/{id} Obter meta por ID

PUT /metas/{id} Atualizar meta

DELETE /metas/{id} Apagar meta

Utilizadores

GET /users Listar utilizadores

GET /users/me Ver perfil do utilizador autenticado

POST /users/me/api-key Regenerar API key

PUT /users/{id}/role Alterar role de um utilizador - apenas admin

Autenticação

O sistema aceita múltiplas formas de autenticação:

- *OAuth2* - Google e Github
- *X-API-Key* - API Key Authentication
- *Basic Authentication*

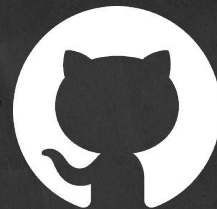


OAuth 2

OAuth2 permite autenticar com provedores externos *sem guardar passwords* desses serviços.

No projeto usamos *Github* e *Google* com os seguintes *callbacks*:

- <http://localhost:3000/auth/google/callback>
- <http://localhost:3000/auth/github/callback>



API Key

O projeto lê a chave no *header X-API-Key*.

Middleware *ensureApiKey*:

- Valida presença da chave
- Procura utilizador na BD (*Utilizador.findOne*)
- Insere *req.user*

Permite autenticação sem sessão/cookie.

Endpoint para regenerar a key: */users/me/api-key*



Basic Auth

Basic Auth usa header *Authorization: Basic base64(user:pass)* .

A nível do *middleware ensureBasicAuth*:

- Decodifica-se as credenciais
- Valida-se o utilizador e *password* associando o utilizador ao *req.user*
- Usa *bcryptjs* para comparar password enviada com *passwordHash* (*bcrypt.compare(plain, hash)*).





Logs de Autenticação

OAuth 2 Login

Provider : google

Username : felipe_castilho

OAuth 2 Login

Provider : github

Username : a047152

API Key Login

ID : 5

Username : cliente

Email : cliente@gym.pt

Role : cliente

Basic Auth Login

Username : cliente

Email : cliente@gym.pt

Passport

Sessões (session/cookie):

No *src/index.js*, o *express-session* está configurado e integrado com o *Passport* (*passport.initialize()* + *passport.session()*).

Depois do *login*, o *Passport* guarda o utilizador na sessão e usa *cookie* para identificar a sessão nos pedidos seguintes.

```
app.use(session({
  secret:      process.env.SESSION_SECRET || 'project_gym_session_secret',
  resave:      false,
  saveUninitialized: false,
  cookie:      { secure: false }
}));

app.use(passport.initialize());
app.use(passport.session());
```


Passport

Sessões (session/cookie):

No *login local* (*passport-local*), quando *req.login(user, ...)* é chamado, o *Passport* salva o *id* na sessão; nos próximos pedidos com o *cookie* da sessão, ele reconstrói *req.user* com *passport.deserializeUser()*.

```
// Guarda só o id na sessão para manter cookie leve.
passport.serializeUser((user, done) => done(null, user.id));

passport.deserializeUser(async (id, done) => {
  try {
    // Reconstrói o user completo a partir do id da sessão.
    const user = await Utilizador.findPk(id);
    done(null, user);
  } catch (err) {
    done(err);
  }
});
```

Autenticação no Postman



Docs Params **Authorization** Headers (8) Body Scripts Settings

Auth Type
Basic Auth

The authorization header will be automatically generated when you send the request. Learn more about [Basic Auth](#) authorization.

Username
treinador

Password
.....

Docs Params **Authorization** Headers (8) Body Scripts Settings

Auth Type
Inherit auth from parent

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#).

API Key

Key
X-API-Key

Value
{{apiKey}}

Add to
Header



Conclusão

Este projeto implementa uma *API REST* com uma camada de autenticação e autorização para acesso aos recursos, baseada na aplicação de ***Basic Auth***, ***API Key*** e ***OAuth2 (Google e GitHub)***.

Na arquitetura, as *routes (endpoints)*, *controllers* (fluxo de dados) e *models* (entidades e as suas relações através do *Sequelize*) permitem a separação das responsabilidades. O *middleware* valida a identidade e atribuir as permissões por *role* do utilizador.
